Uppsala University



Machine Learning

Assignment 7

General information

- The recommended tool in this course is R (with the IDE R-Studio). You can download R **here** and R-Studio **here**. You can use Python and Jupyter Notebooks, although the assignments may use data available only through the R package, a problem you would need to solve yourself.
- Report all results in a single, `*.pdf`-file. *Other formats, such as Word, Rmd, Jupyter Notebook, or similar, will automatically be failed.* Although, you are allowed first to knit your document to Word or HTML and then print the assignment as a PDF from Word or HTML if you find it difficult to get TeX to work.
- The report should be written in English.
- If a question is unclear in the assignment. Write down how you interpret the question and formulate your answer.
- You should submit the report to **Studium**. Deadlines for all assignments are **Sunday 23.59**. See **Studium** for dates. Assignments will be graded within 10 working days from the assignment deadline.
- To pass the assignments, *you should answer all questions not marked with **, and get at least 75% correct.
- To get VG on the assignment, *also the questions marked with a ** should be answered. Sometimes you can choose between different VG assignments, then this is explicitly stated. To get VG you need 75% both on questions to pass and the VG part of the assignment. VG will only be awarded on the first deadline of the assignment.
- A report that does not contain the general information (see the **template**), will be automatically rejected.
- Every assignment report must include one oral exam question of your own together with a short model answer, following the oral exam question contribution instructions in this assignment. You are not allowed to use large language models, such as ChatGPT, at all for this contribution.
- State clearly whether the course staff may use or adapt your question and answer in a future oral exam: **Yes** or **No**.
- When working with R, we recommend writing the reports using R markdown and the provided **R markdown template**. The template includes the formatting instructions and how to include code and figures.
- Instead of R markdown, you can use other software to make the pdf report, but you should use the same instructions for formatting. These instructions are also available in **the PDF produced from the R markdown template**.
- The course has its own R package `uuml` with data and functionality to simplify coding. To install the package just run the following:

```
1. install.packages("remotes")
```

2. `remotes::install_github("MansMeg/IntroML", subdir = "rpackage")`

- We collect common questions regarding installation and technical problems in a course Frequently Asked Questions (FAQ). This can be found **here**.
- You are not allowed to show your assignments (text or code) to anyone. Only discuss the assignments with your fellow students. The student that show their assignment to anyone else could also be considered to cheat. During scheduled support sessions held on Zoom, only share your screen in a separate room with a teaching assistant.
- You may use large language models (LLMs), such as ChatGPT, like you may discuss with fellow students: to learn, ask for explanations, and get feedback. You may not use LLMs to produce the solution you submit. You must be able to explain every part of your own assignment at the oral exam, including your code, results, and reasoning, even when it is not covered by the published oral exam questions. If course staff have concerns about your understanding or the authorship of any part of your submission, you may be asked to explain that part at the oral exam.
- Where an exercise explicitly asks you to run an LLM and report its output, you may do so for that experiment. Your explanations and analysis must be your own. This permission does not apply to the oral exam question contribution, for which LLM use is prohibited entirely.
- Do not copy or upload the full assignment text into an LLM.
- Scheduled support sessions are for questions about the assignments and course material. Do not hesitate to ask! Ask questions early if instructions are unclear or you encounter possible bugs, so you do not lose time unnecessarily.
- If you have any suggestions or improvements to the course material, please post in the course chat feedback channel, create an issue **here**, or submit a pull request to the public repository.

Contents

Oral exam question contribution 4

1 Variational Autoencoders 5

2 Topic Models 7

3 * Variational Autoencoders using Convolutional Neural Networks 10

4 * Prompt engineering with large language models 11

4.1 Make a language model hallucinate 13

4.2 In-Context Learning for few-shot classification 15

Reading priorities for the basic assignment.

- **VAE:** Read Bishop and Bishop (2024, Section 19.2). Prioritize Kingma and Welling (2019, Sections 2.1–2.5) for additional detail on the encoder/decoder, ELBO (reconstruction and KL terms), and Gaussian reparameterization.
- **Topic models:** Focus on *Using Gibbs Sampling to Discover Topics* in Griffiths and Steyvers (2004) for the model and sampler. Use *Latent Dirichlet Allocation* in Blei (2012) for an introductory overview.

Oral exam question contribution

Write one possible oral exam question of your own for this assignment together with a short model answer. Include both in your assignment report.

The question should:

- concern material from this assignment or the associated reading or lectures,
- be suitable for a 2–4 minute oral answer, possibly using a whiteboard,
- test conceptual understanding rather than only calculation or code,
- not already be included in the oral exam question list and not be very close to an existing question (e.g. just different numbers).

Note! You are not allowed to use large language models, such as ChatGPT, at all for this oral exam question contribution.

State clearly whether the course staff may use or adapt your question and answer in a future oral exam: **Yes** or **No**.

1 Variational Autoencoders

Keras, TensorFlow, and Google Colab

This assignment uses `keras3` and `tensorflow` for R. Use the `keras3` package for all Keras code. Adam is the default optimizer: use `optimizer_adam(learning_rate = 0.001)` unless the task or supplied example specifies another learning rate or explicitly compares optimizers. To work in Google Colab, open the course R setup notebook, save a copy in Google Drive, and run its setup and training-check cells. Wait for "PASS: training and prediction work" before starting the assignment.

Use the assignment report template linked in the notebook to structure your report. Write explanations in text cells and put R code in code cells. Save your notebook and results in Drive, and run the setup cell again at the start of each R session.

For local installation, follow the course R setup guide. Details on the architecture components are available in the `keras3` reference.

We will implement a variational autoencoder in R using `keras3` and TensorFlow. Use the feed-forward VAE example as a starting point.

Note! Training can take time. Run it once and reuse the fitted encoder and decoder for the remaining tasks. Keep the training output and figures for your report.

1. (1p) Start by loading the MNIST data in R. See the link above or previous assignments for details.

2. (1p) Now, implement a one-layer (encoder and decoder) feed-forward variational autoencoder with two latent dimensions. Both the encoder layer and the decoder layer should have 200 hidden units. You should end up with a variational autoencoder with roughly 310 000 - 320 000 parameters.

   Example: Sanity check *No submission required*

   For a batch of m images, the encoder should return an $m \times 2$ matrix of latent means. Decoding these should return an $m \times 784$ matrix with pixel values in $[0, 1]$. Check a few images before full training.

3. Print the model and include it in your report.

   (a) (1p) How many weights (parameters) are used to compute μ and σ^2 for the latent variables?

   (b) (1p) **KL experiment.** The function below computes $D_{\mathrm{KL}}(q\|p)$, comparing the encoder distribution $q = N(\mu, \sigma^2)$ with the prior $p = N(\mu_0, \sigma_0^2)$. Both σ and σ_0 are standard deviations. Keep the prior fixed at $(\mu_0, \sigma_0) = (0, 1)$. Predict the ordering for q with $(\mu, \sigma) = (0, 1)$, $(2, 1)$, and $(0, 2)$, then calculate the KL values. Which q is closest to the prior? Why would minimizing only KL fail to reconstruct different images?

   Hint! For this fixed prior, the function matches the one-dimensional KL term in the sampling layer.

The formula is

$$D_{\mathrm{KL}}(q\|p) = \log \frac{\sigma_0}{\sigma} + \frac{\sigma^2 + (\mu - \mu_0)^2}{2\sigma_0^2} - \frac{1}{2}.$$

The function is available in `uuml` version 0.5.1 or later; reinstall the package using the general instructions if needed. Its implementation is printed below. Use `?kl_normal` for documentation.

```
kl_normal <- function (mu, sigma, mu_0, sigma_0)
{
    checkmate::assert_numeric(mu, finite = TRUE, any.missing = FALSE)
    checkmate::assert_numeric(sigma, finite = TRUE, any.missing = FALSE)
    checkmate::assert_numeric(mu_0, finite = TRUE, any.missing = FALSE)
    checkmate::assert_numeric(sigma_0, finite = TRUE, any.missing = FALSE)
    if (any(sigma <= 0) || any(sigma_0 <= 0)) {
        stop("sigma and sigma_0 must be positive standard deviations.")
    }
    sizes <- vapply(list(mu, sigma, mu_0, sigma_0), length, integer(1))
    if (any(!sizes %in% c(1L, max(sizes)))) {
        stop("Arguments must have length one or a common length.")
    }
    log(sigma_0/sigma) + (sigma^2 + (mu - mu_0)^2)/(2 * sigma_0^2) -
        0.5
}
```

```
uuml::kl_normal(mu = -1, sigma = 0.5, mu_0 = 0, sigma_0 = 1)
```

4. Before training, use the supplied `vae_loss_components()` scaffold to record the reconstruction, KL, and total losses. Train your variational autoencoder on MNIST for 50 epochs, then record them again using the same 100 images and the same sampled noise. The example stores these as `loss_before` and `loss_after`. Visualize the latent state for the different numbers.

   (a) (1p) How do you interpret this latent state?

   (b) (1p) What numbers are better represented by the latent state?

   (c) (1p) What number is less well represented by the latent state?

   (d) (1p) **Loss experiment.** Report the before/after loss table. Which term changed most? Do reconstruction and KL loss both need to decrease for their sum to improve? Explain using your results.

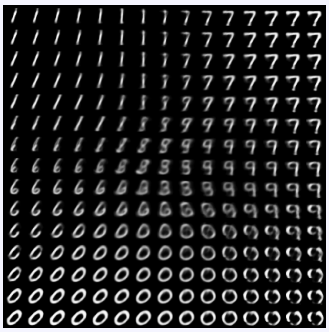
5. (2p) Finally, encode all the 2:s in the MNIST test dataset to the latent state using your encoder. What is the mean of the digits "2" in the two latent dimensions? *Hint!* See `y_test` for the MNIST numbers. *Note!* Do not retrain your VAE. Just use the one you have already trained.

6. (1p) Visualize this value of the latent state as a 28 by 28-pixel image using your decoder.

Example: Inspecting the latent grid *No submission required*

After training, use `uuml::plot_vae_grid(generator)` to inspect images decoded from a grid of two-dimensional latent coordinates. Here `generator` is your trained decoder, as named in the supplied example. The helper works with both the feed-forward and convolutional VAE examples and uses their existing weights. It does not train another model. Use `?plot_vae_grid` to see how to choose the coordinate grid and image dimensions.

Example: VAE reference output *No submission required*



Illustrative decoder grid from an earlier training run of the feed-forward VAE after 50 epochs on MNIST. Each cell decodes a different latent coordinate. Some digits may look blurred or blended. Your arrangement can differ because training is stochastic and the latent axes have no fixed interpretation.

2 Topic Models

We will now analyze the classical book Pride and Prejudice by Jane Austen using a probabilistic topic model. If you have not read the book, **here** you can read up on the story of this classical book.

We will use a Gibbs sampler to estimate ten different topics occurring in Pride and Prejudice and study where they occur. A tokenized version of the book and a `data.frame` with stopwords can be loaded as follows:

```
library(uuml)
library(dplyr)
library(tidytext)
data("pride_and_prejudice")
data("stopwords")
```

1. **Setup (ungraded).** Remove stopwords (common English words without much semantic information):

```
pap <- pride_and_prejudice
pap <- anti_join(pap, y = stopwords[stopwords$lexicon == "snowball",])

## Joining with `by = join_by(word)`
```

2. **Setup (ungraded).** Remove words that occur at most five times:

```
word_freq <- table(pap$word)
rare_words <- data.frame(word = names(word_freq[word_freq <= 5]),
                         stringsAsFactors = FALSE)
pap <- anti_join(pap, y = rare_words)

## Joining with `by = join_by(word)`
```

3. (1p) The supplied code builds a document-term matrix, treating each paragraph as a document. Report the number of documents and terms (word types).

```
library(tm)
crp <- aggregate(pap$word, by = list(pap$paragraph),
                 FUN = paste0, collapse = " ")
names(crp) <- c("paragraph", "text")
s <- SimpleCorpus(VectorSource(crp$text))
m <- DocumentTermMatrix(s)
```

4. **Setup (ungraded).** Fit a topic model with ten topics using Gibbs sampling for 2000 iterations and the fixed seed below. Fit the model once and reuse the fitted model `tm` for the likelihood trace, topic interpretation, and paragraph plots.

```
library(topicmodels)
K <- 10
# Note: delta is beta in Griffith and Steyvers (2004) notation.
control <- list(keep = 1, delta = 0.1, alpha = 1, iter = 2000,
                seed = 4711)
tm <- LDA(m, k = K, method = "Gibbs", control)
```

5. **Setup (ungraded).** Extract Θ, Φ, and the log-likelihood values using the supplied code. This is the parameter notation used in Griffiths and Steyvers (2004).

```
library(uuml)
lls <- extract_log_liks(tm)
theta <- extract_theta(tm)
phi <- extract_phi(tm)
```

6. (2p) Plot the log-likelihood against Gibbs iterations. Does it settle into a roughly stable range, or does a persistent trend remain? Support your assessment using the plot.

   Hint! Fluctuations are normal in Gibbs sampling. Look for changes in the typical level rather than expecting a smooth or steadily increasing curve. A single likelihood trace cannot establish convergence of all model parameters.

7. (2p) Inspect the 20 top words for each topic and choose two whose words seem to belong together. For each chosen topic, inspect one or two paragraphs where its estimated topic proportion is high. Include the two word lists, suggest an informative label for each topic, and explain each label using the top words and paragraph content.

   Hint! Set `topic` to a chosen topic number. Use `crp$paragraph` to map rows of `theta` back to the original text.

```
topic <- 1
paragraph_ids <- crp$paragraph[
  head(order(theta[, topic], decreasing = TRUE), 2)]
pride_and_prejudice |>
  filter(paragraph %in% paragraph_ids) |>
  group_by(paragraph) |>
  summarise(text = paste(word, collapse = " "), .groups = "drop")
```

8. (2p) Visualize these two topics evolve over the paragraphs in the books by plotting the θ parameters for that topic over time (paragraphs) in the book. Think of this as the time-line of the book. On the y-axis, you should plot θ_i for your chosen topic i and the x-axis should be the paragraph number (first paragraph has number 1 and so forth).

9. (2p) How do these two chosen topics evolve over the course in the book? If you want, you can take a rolling mean of the theta parameters to more easily show the changes in the topic over the book. *Hint!* Here `zoo::rollmean()` might be a good function to use.

3 * Variational Autoencoders using Convolutional Neural Networks

This task is only necessary to get one point toward a *pass with distinction* (VG) grade. Hence, if you do not want to get a *pass with distinction* (VG) point, you can ignore this part of the assignment.

You can choose to either do this task or the task below to get VG on this assignment.

As we have seen previously, for images, we can get better performance using Convolutional Neural Networks. Hence we are going to repeat the exercise above using a convolutional neural network as encoder and decoder. You can find detailed code in the file `variational_autoencoder_deconv_keras3.R`.

1. (3p) Now implement a four-layer (encoder and decoder) convolutional neural network with two latent dimensions. Use 50 filters in each hidden convolutional layer. *Note!* A dense layer should be included as the last step in the encoder and the first step in the decoder. These layers should have 100 hidden units. You should end up with a variational autoencoder with roughly 2M parameters.

2. (1p) Print the model and include it in your report. How many weights (parameters) are used to compute μ and σ^2 for the latent variables? What layer represent the latent variables?

3. (1p) Now train your CNN variational autoencoder on the MNIST data for five epochs. Visualize the latent state for the different numbers. How do you interpret this result? Compare these results with the results from the feed-forward autoencoder.

4. (1p) Finally, encode all the 2:s in the MNIST test dataset to the latent state using your encoder (Hint!, see `y_test` for numbers). What is the mean of the digits "2" in the two latent dimensions?

5. (1p) Visualize the mean value of the digit 2 of the latent state as a 28 by 28-pixel image using your decoder.

4 * Prompt engineering with large language models

You can choose to either do this task or the task above to get VG on this assignment.

This task is only necessary to get one point toward a *pass with distinction* (VG) grade. Hence, if you do not want to get a *pass with distinction* (VG) point, you can ignore this part of the assignment.

For this task, choose either **OpenAI** or **Anthropic (Claude)**. You only need a developer account and an API key with your chosen provider. Follow the OpenAI API quickstart or the Claude API quickstart. API billing is separate from consumer chat subscriptions; check the current OpenAI API prices or Claude API prices before running your experiments.

Install `ellmer` from CRAN and open your personal `.Renviron` file:

```
install.packages("ellmer")
file.edit(path.expand("~/.Renviron"))
```

Add *only the line for your chosen provider*, replacing the placeholder with your own API key, then save the file and restart R:

- OpenAI: `OPENAI_API_KEY=YOUR_API_KEY`
- Anthropic: `ANTHROPIC_API_KEY=YOUR_API_KEY`

`ellmer` reads the corresponding environment variable automatically. Select your provider below; the check confirms that a key has been set without printing it:

```
library(ellmer)
provider <- "openai"  # Change to "anthropic" for Claude.
key_name <- switch(provider,
  openai = "OPENAI_API_KEY",
  anthropic = "ANTHROPIC_API_KEY",
  stop("Choose 'openai' or 'anthropic'.")
)
stopifnot(nzchar(Sys.getenv(key_name)))
```

Warning: Keep your API key private. Do not put it in your R scripts or report, and do not upload your `.Renviron` file to Github, public Colab notebooks, or other shared locations.

Choosing the baseline model. Start with the cheapest currently available general-purpose text model that you can access through your chosen provider's API. Define "cheapest" using the published standard price for 1,000 input tokens and 1,000 output tokens, excluding caching and batch discounts. Consult the OpenAI model list or Claude model list and the corresponding pricing page. Use this baseline for the initial hallucination experiment and the few-shot classification task below.

Replace `YOUR_MODEL_ID` with its exact model ID. Always specify a model explicitly because `ellmer`'s defaults can change.

```
model_id <- "YOUR_MODEL_ID"  # Replace with your baseline model ID.

new_chat <- function(model = model_id) {
  if (provider == "openai") {
    chat_openai(model = model, echo = "none")
  } else {
    chat_anthropic(model = model, echo = "none", cache = "none")
  }
}

ask_llm <- function(prompt, model = model_id) {
  chat <- new_chat(model)
  as.character(chat$chat(prompt))
}

response <- ask_llm("Who is Gustav Vasa? Give a short answer.")
cat(response)
```

An illustrative response is shown below; the actual wording can differ.

```
## Gustav Vasa was a Swedish king who led a successful rebellion
## against Danish rule and became the founder of modern Sweden in
## the 16th century.
```

Each call to `ask_llm()` starts a fresh conversation. Use it for independent prompts and for each quote in the classification task below, so earlier quotes and responses do not affect later predictions.

For a deliberate multi-turn experiment, create one chat object and reuse it. Here, the second prompt includes the first exchange as context:

```
chat <- new_chat()
first_response <- chat$chat("Who is Gustav Vasa? Give a short answer.")
follow_up <- chat$chat("Which century did he live in?")
cat(follow_up)
```

See `?chat_openai`, `?chat_anthropic`, and the ellmer documentation for further details. If a request reports insufficient quota, check your chosen provider's billing and available API credits.

4.1 Make a language model hallucinate

We are now going to understand how and when transformer-based decoder models 'hallucinate', i.e. when the models give factually incorrect information. See Zhao et al. (2023, Section 7.1.2) for an introduction and Huang et al. (2023) for details on hallucinations in LLMs.

Investigate three distinct strategies for eliciting factual errors using your baseline model. Make **at most three API calls per strategy**, including follow-up messages. Stop that strategy when you identify a factual error or reach the limit. Each follow-up message counts as another call.

Select one test per strategy for your report and the model comparison below: the first test that produced a verified factual error, or your final test if no error occurred. For a multi-turn test, keep the full conversation leading to the selected response.

No hallucination is also a valid result. Correct answers, refusals, and explicit uncertainty are acceptable outcomes to analyse. You are assessed on your experimental design, factual verification, and reasoning; you do not need to force the model to make an error or exceed the call limit.

For each strategy, report:

1. The strategy and the number of API calls used.
2. The selected test's full prompt or conversation history, generation settings, and model response. Record a seed if the API supports one; otherwise state that it is unavailable.
3. Whether the response contains a factual error, supported by a reliable source or the supplied context. If it does, explain whether it is an intrinsic or extrinsic hallucination. A refusal or an expression of uncertainty alone is not a hallucination.
4. Your analysis of why the strategy did or did not elicit a hallucination.

Comparing with a stronger model. Select a second model from the *same provider* that the provider describes as more capable than your baseline. Repeat the three selected tests, including tests where the baseline did not hallucinate, starting a fresh conversation for each independent test. For a multi-turn test, replay the same sequence of user prompts in a new conversation with the stronger model. Keep generation settings comparable where supported and report both transcripts. Make no additional attempts to find errors in the stronger model: this comparison uses at most nine calls, giving at most 18 calls across both models for this subsection.

For a single-prompt test, set `stronger_model_id` to the stronger model's exact ID and use `ask_llm(prompt, model = stronger_model_id)`. For a multi-turn test, start with `new_chat(model = stronger_model_id)`.

Report your provider, both exact model IDs, the date of your experiments, and the published input and output token prices, with a link to the pricing page. Explain why you chose the stronger model. Then discuss:

1. The stronger model's response to each test. Is it still factually incorrect? Does it express uncertainty or correct the mistaken premise? If you find no hallucinations in these responses, state that.

2. How factual accuracy, hallucinations, and uncertainty differ between the two models. Does the stronger model perform better on your examples, and would the difference justify its higher cost?

4.2 In-Context Learning for few-shot classification

In-context learning uses examples in the prompt to guide a language model without changing its weights. Here, classify Swedish political quotes as coming from the Social Democrats (`S`) or Moderaterna (`M`). Use the same baseline model and generation settings throughout this experiment.

Developing and validating prompts. Use only `pc_train` to develop your prompts. The following fixed, stratified split provides six quotes for few-shot demonstrations and four different quotes for validation. Keep these sets separate: a validation quote must never appear among the demonstrations used to classify it.

Note! These data are included in `uuml` version 0.4.0 and later.

```
library(uuml)
data("pc_train")
set.seed(4711)
example_id <- unlist(lapply(
  split(seq_len(nrow(pc_train)), pc_train$party),
  function(i) sample(i, 3L)
), use.names = FALSE)
pc_examples <- pc_train[example_id, ]
pc_validation <- pc_train[-example_id, ]
```

Keep `pc_test` untouched during development: do not inspect its quotes or labels, use them as demonstrations, or use test results to revise your prompts.

For a few-shot prompt, combine the demonstration quotes and their labels into a string:

```
example_string <- paste0(pc_examples$quote,
                         ": ",
                         pc_examples$party, collapse = "\n\n")
cat(example_string)
```

Example: Prompt formatting *No submission required*

```
instruction <- paste(
  "Classify the quote as M (Moderaterna) or S (Social Democrats).",
  "Return exactly M or S, with no explanation."
)
prompt <- paste(
  instruction,
  paste("Examples:", example_string, sep = "\n"),
  paste("Quote to classify:", pc_validation$quote[1], sep = "\n"),
  sep = "\n\n"
)
cat(prompt)
```

`"\n"` starts a new line; `"\n\n"` separates blocks with a blank line. Inspect the prompt with `cat(prompt)`. For zero-shot classification, omit the examples block.

Use the techniques from Zhao et al. (2023, Sections 6.1–6.3) to develop three distinct prompt strategies:

1. A simple zero-shot baseline with no demonstrations.
2. An improved zero-shot prompt with a clearer task description or instructions.
3. A few-shot prompt using demonstrations drawn only from `pc_examples`.

Evaluate candidate prompts on the same four validation quotes. You may revise them using the validation results. Explain the changes you made and select one final prompt for each of the three strategies.

Freeze the experiment before testing. Fix all three final prompt templates, any demonstration selection and ordering, the model ID, generation settings, and output-scoring rule. Save these choices before loading the test data. Do not revise them after seeing test responses or accuracy.

Only then load `pc_test` and evaluate all three frozen strategies on its same 20 quotes. This requires one response per quote and strategy (60 test responses in total). Start a fresh chat for each quote, for example using `ask_llm()`, and include only the quote text as the test input; use `pc_test$party` solely for scoring after predictions have been generated. Save the raw responses so you can analyse them without repeating API calls.

```
data("pc_test")
```

Output and scoring. Require exactly `M` or `S`. Remove only leading and trailing whitespace with `trimws()`. Treat any other response, including an explanation, refusal, empty response, or missing prediction, as invalid and incorrect; do not extract a label from a longer answer or drop invalid predictions. For both validation and test data, calculate

$$\text{accuracy} = \frac{\text{number of correct, valid predictions}}{\text{total number of quotes}}.$$

The test denominator is always 20 for each strategy.

Include the following in your report:

1. Your three final prompt templates, demonstration row indices from `pc_train` and their order, model ID, and generation settings. Describe how validation informed your choices.
2. A comparison table with one row per strategy: validation accuracy (out of 4), test accuracy (out of 20), and the number of invalid test responses. Include the raw responses and scored predictions in your submitted code or supporting files; you do not need to repeat all quotes in the report.
3. Whether clearer instructions and few-shot examples improved the results, supported by the table and selected responses. Discuss the limitations of drawing conclusions from these small datasets. An improvement in accuracy is not required; assess what your results show.

References

Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. doi: 10.1007/978-3-031-45468-4. URL https://www.bishopbook.com/.

Diederik P. Kingma and Max Welling. An introduction to variational autoencoders. *arXiv preprint arXiv:1906.02691*, 2019. URL https://arxiv.org/abs/1906.02691.

Thomas L Griffiths and Mark Steyvers. Finding scientific topics. *Proceedings of the National academy of Sciences*, 101(suppl 1):5228–5235, 2004.

David M Blei. Probabilistic topic models. *Communications of the ACM*, 55(4):77–84, 2012.

Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models, 2023.

Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions, 2023.